

Retail Data Engineering – Ingestion & Medallion Processing

Azure Data Factory • ADLS Gen2 • Databricks

Project focus: End-to-end retail data ingestion and processing using multiple source systems, Azure Data Factory, ADLS, and Databricks. The project requirements specify transaction, store, and product data in Azure SQL and customer data in API JSON format, with the data brought into the data lake.

Implementation Flow

Source Data → Azure Data Factory → ADLS Bronze → Databricks Transformation → ADLS Silver → Gold Curated Data

Each following page contains one screenshot extracted from the uploaded project PDF at the **same display size as the original screenshot**, followed by a separate explanation for that screenshot.

Step 1: Verify the ADLS Storage Structure

Home > kpstorage898_1790312486474 | Overview > kpstorage898

kpstorage898 | Containers ☆ ...

Storage account

Search

+ Add container ↑ Upload ↻ Refresh | 🗑 Delete 🔒 Change access level ↺ Restore containers ▾ 🛠 Edit columns

Search containers by prefix Only show active containers ▾

Showing all 4 items

☐	Name	Last modified	Anonymous access level	Lease state	
☐	\$logs	9/25/2026, 10:32:02 AM	Private	Available	...
☐	retail-bronze	9/25/2026, 10:35:31 AM	Private	Available	...
☐	retail-gold	9/25/2026, 10:35:50 AM	Private	Available	...
☐	retail-silver	9/25/2026, 10:35:40 AM	Private	Available	...

Overview
Activity log
Tags
Diagnose and solve problems
Access Control (IAM)
Data migration
Events
Storage browser
Partner solutions
Resource visualizer
Data storage
Containers
Classic file shares
Queues
Tables
Security + networking
Add or remove favorites by pressing Ctrl+Shift+F

Azure Storage account and Bronze, Silver, and Gold containers

Step-by-step explanation

- Open the Azure Storage account and navigate to the Containers section.
- Verify that the storage account contains the project containers used for the data pipeline.
- The Bronze layer is used for incoming/raw data, while Silver and Gold provide progressively processed data areas.
- Confirm that the required containers are available before starting ingestion and transformation.

Step 2: Review the Product Source Data

Microsoft Azure

Search resources, services, and docs (G+)

Copilot

krishnaprasadpm758@...
DEFAULT DIRECTORY (EVANPUNL...)

Home > kpdatabase (kpsqlserver/kpdatabase)

kpdatabase (kpsqlserver/kpdatabase) | Query editor (preview)

SQL database

Search

New query Templates Open in Feedback Sign out Classic experience

Overview
Activity log
Tags
Diagnose and solve problems
Query editor (preview)
Mirror database in Fabric (preview)
Resource visualizer
Settings
Data management
Integrations
Power Platform
Security
Intelligent performance
Monitoring
Automation

kpdatabase

dbo

Tables

products

stores

transactions

Views

Stored Proced...

Functions

Queries

SQL query 1

SQL query 1 SQL query 2

Run Save as Save as view Download .sql

1 SELECT TOP (1000) * FROM [dbo].[products]

Messages Results Download as .csv Download as .json Download results (as .xlsx)

	product_id	product_name	category	price
1	1	Wireless Mouse	Electronics	799.99
2	2	Bluetooth Speaker	Electronics	1299.49
3	3	Yoga Mat	Fitness	499.00
4	4	Laptop Stand	Accessories	999.99
5	5	Notebook Set	Stationery	149.00
6	6	Water Bottle	Fitness	299.00

Succeeded (1 sec 175 ms)

Columns: 4 Rows: 10

Add or remove favorites by pressing Ctrl+L+Shift+F

Azure SQL product table

Step-by-step explanation

- Open the Azure SQL database in the Query Editor.
- Run a query against the **products** table to inspect the source records.
- Verify the product fields shown in the result, including product ID, product name, category, and price.
- This source data is required for the retail pipeline because product data is one of the SQL-based source datasets described in the project requirements.

Step 3: Review the Store Source Data

Microsoft Azure

Search resources, services, and docs (G+)

Copilot

krishnaprasadpm758@...
DEFAULT DIRECTORY (EVANPUNL...

Home > kpdatabase (kpsqlserver/kpdatabase)

kpdatabase (kpsqlserver/kpdatabase) | Query editor (preview)

SQL database

Search

New query, Templates, Open in, Feedback, Sign out, Classic experience

Overview, Activity log, Tags, Diagnose and solve problems, Query editor (preview), Mirror database in Fabric (preview), Resource visualizer, Settings, Data management, Integrations, Power Platform, Security, Intelligent performance, Monitoring, Automation

kpdatabase

- dbo
 - Tables
 - products
 - stores
 - transactions
 - Views
 - Stored Proced...
 - Functions
- Queries
 - SQL query 1

SQL query 1, SQL query 2, SQL query 3

Run, Save as, Save as view, Download .sql

1 SELECT TOP (1000) * FROM [dbo].[stores]

Messages, Results, Download as .csv, Download as .json, Download results (as .xlsx)

	store_id	store_name	location
1	1	City Mall Store	Mumbai
2	2	High Street Store	Delhi
3	3	Tech World Outlet	Bangalore
4	4	Downtown Mini Store	Pune
5	5	Mega Plaza	Chennai

Succeeded (0 sec 118 ms) Columns: 3 Rows: 5

Add or remove favorites by pressing Ctrl+Shift+F

Azure SQL store table

Step-by-step explanation

- Open the **stores** table in the Azure SQL Query Editor.
- Run the SELECT query and inspect the returned store records.
- Verify the store identifier, store name, and location values.
- This validates the store master data before it is brought into the data lake.

Step 4: Review the Transaction Source Data

Microsoft Azure

Search resources, services, and docs (G+/)

Copilot

krishnaprasadpm758@...
DEFAULT DIRECTORY (EVANPUNL...

Home > kpdatabase (kpsqlserver/kpdatabase)

kpdatabase (kpsqlserver/kpdatabase) | Query editor (preview)

SQL database

Search

New query, Templates, Open in, Feedback, Sign out, Classic experience

Overview, Activity log, Tags, Diagnose and solve problems, Query editor (preview), Mirror database in Fabric (preview), Resource visualizer, Settings, Data management, Integrations, Power Platform, Security, Intelligent performance, Monitoring, Automation

kpdatabase > dbo > Tables > products, stores, transactions > Views > Stored Proced... > Functions > Queries > SQL query 1

SQL query 1, SQL query 2, SQL query 3, SQL query 4

Run, Save as, Save as view, Download .sql

1 SELECT TOP (1000) * FROM [dbo].[transactions]

Messages, Results, Download as .csv, Download as .json, Download results (as .xlsx)

	123	transaction_id	123	customer_id	123	product_id	123	store_id	123	quantity	transaction_date
1	1		127		8		4		4		2025-03-31
2	2		105		3		4		5		2024-11-12
3	3		116		2		2		3		2025-05-01
4	4		120		8		1		1		2024-11-02
5	5		105		5		2		1		2025-03-17
6	6		110		7		3		5		2025-01-04

Succeeded (0 sec 126 ms) Columns: 6 Rows: 30

Azure SQL transaction table

Step-by-step explanation

- Open the **transactions** table in the Azure SQL Query Editor.
- Execute the SELECT query and inspect the transaction records.
- Verify the transaction ID, customer ID, product ID, store ID, quantity, and transaction date columns.
- The project requirements identify transaction data as one of the primary Azure SQL source datasets.

Step 5: Configure the Linked Services

Microsoft Azure | Data Factory | kpdatafactory45457

Search

krishnaprasadpm758@gmail.com
DEFAULT DIRECTORY

Data Factory | Validate all | Publish all

General

- Connector upgrade advi...
- Factory settings

Connections

- Linked services**
- Integration runtimes
- Microsoft Purview
- ADF in Microsoft Fabric

Source control

- Git configuration
- ARM template

Author

- Triggers
- Global parameters
- Data flow libraries

Security

- Credentials
- Customer managed key
- Outbound rules
- Managed private endpoi...

Linked services

Linked service defines the connection information to a data store or compute. [Learn more](#)

+ New

Filter by name Annotations : Any

Showing 1 - 3 of 3 items

Name ↑↓	Type ↑↓	Related ↑↓	Annotations ↑↓
LS_ADLS	Azure Data Lake Storage Gen2	0	
LS_REST	REST	0	
LS_SQL	Azure SQL Database	0	

ADF linked services for ADLS, REST, and Azure SQL

Step-by-step explanation

- Open the Data Factory Manage area and select **Linked services**.
- Verify the configured connections for the ADLS storage account, REST API, and Azure SQL Database.
- These connections provide the pipeline with access to the different source and destination systems.
- The REST connection is used for API-based data, while Azure SQL supplies the relational retail data.

Retail Data Engineering • Screenshot 5 of 18

Step 6: Configure the Transaction Copy Activity

The screenshot displays the Microsoft Azure Data Factory (ADF) console. The top navigation bar shows the user's profile and email. The left sidebar contains the 'Factory Resources' pane with a search bar and a list of resources including Pipelines, Datasets, and Data flows. The main workspace shows a pipeline named 'PL_Ingest_Retail_Data' with four 'Copy data' activities connected in sequence. The first activity, 'Copy data1', is selected, and its configuration pane is open. The 'Source' tab is active, showing the 'Source dataset' as 'DS_SQL_TRANSACTIONS'. The 'Use query' section has 'Table' selected. The 'Query timeout (minutes)' is set to 120. The 'Isolation level' is set to 'Select...'. The 'Partition option' is set to 'None'. A message at the bottom of the configuration pane says 'Please preview data to validate the partition settings.' Below the configuration pane, the text 'ADF pipeline with the transaction source dataset' is visible.

Step-by-step explanation

- Open the retail ingestion pipeline in Azure Data Factory.
- Select the Copy Data activity connected to the transaction source.
- Configure the source dataset for the Azure SQL transaction data and verify the source query/settings.
- Use the ADLS destination configured for the raw/Bronze ingestion stage.

Step 7: Configure the Store Copy Activity

The screenshot displays the Microsoft Azure Data Factory (ADF) console. The top navigation bar shows the user's profile and email. The left sidebar contains the 'Factory Resources' pane with sections for Pipelines, Datasets, Data flows, and Power Query. The main workspace shows a pipeline named 'PL_Ingest_Retail_Data' with four 'Copy data' activities: 'Copy data1', 'Copy data2', 'Copy data3', and 'Copy data4'. The 'Copy data2' activity is selected, and its configuration pane is open. The 'Sink dataset' is set to 'DS_ADLS_STORES'. The 'Copy behavior' is set to 'Select...'. The 'Max concurrent connections' is set to 1. The 'Block size (MB)' is set to 1. The 'Metadata' is set to 'New'. The 'Max rows per file' is set to 1. The 'Sink dataset' dropdown is set to 'DS_ADLS_STORES'. The 'Copy behavior' dropdown is set to 'Select...'. The 'Max concurrent connections' input field is set to 1. The 'Block size (MB)' input field is set to 1. The 'Metadata' dropdown is set to 'New'. The 'Max rows per file' input field is set to 1.

ADF pipeline with the store source and destination

Step-by-step explanation

- Select the Copy Data activity used for the store dataset.
- Verify the source and sink configuration shown in the pipeline.
- Configure the store data to be copied from the source system into the data-lake landing area.
- This activity ensures store master data is available for downstream transformation.

Step 8: Configure the Customer Copy Activity

The screenshot displays the Microsoft Azure Data Factory (ADF) interface. On the left, the 'Factory Resources' pane shows a list of pipelines and datasets. The 'PL_Ingest_Retail_Data' pipeline is selected. The 'Activities' pane on the right shows a sequence of four 'Copy data' activities: 'Copy data1', 'Copy data2', 'Copy data3', and 'Copy data4'. The 'Copy data3' activity is currently selected, and its configuration pane is open. The configuration pane has tabs for 'General', 'Source', 'Sink', 'Mapping', 'Settings', and 'User properties'. The 'Source' tab is active, showing the 'Source dataset' as 'DS_SQL_PRODUCTS'. The 'Use query' option is set to 'Table'. The 'Query timeout (minutes)' is set to 120. The 'Isolation level' is set to 'Select...'. The 'Partition option' is set to 'None'. A message at the bottom of the configuration pane states: 'Please preview data to validate the partition settings.' Below the configuration pane, the text 'ADF pipeline activity for customer/API data' is visible.

ADF pipeline activity for customer/API data

Step-by-step explanation

- Select the Copy Data activity used for customer data.
- Configure the REST/API source connection and the corresponding dataset.
- Verify the sink configuration so the customer data is landed in the data lake.
- The project requirements specify that customer data is received in API JSON format.

Step 9: Configure the Product Copy Activity

The screenshot displays the Microsoft Azure Data Factory (ADF) console. The top navigation bar shows the user's profile and email. The left sidebar contains 'Factory Resources' with a search bar and a list of pipelines and datasets. The 'Pipelines' section is expanded, showing a pipeline named 'PL_Ingest_Retail_Data'. The 'Activities' section is also expanded, showing a list of activities including 'Copy data'. The main canvas shows a pipeline diagram with four 'Copy data' activities: 'Copy data1', 'Copy data2', 'Copy data3', and 'Copy data4'. The 'Copy data4' activity is selected, and its configuration pane is open. The 'Source' tab is active, showing the 'Source dataset' as 'DS_REST_CUSTOMERS'. The 'Request method' is set to 'GET', the 'Request timeout' is '00:01:40', and the 'Request interval (ms)' is '10'. The 'Additional headers' section is empty, and the 'Pagination rules' section is also empty.

ADF pipeline activity for product data

Step-by-step explanation

- Select the Copy Data activity used for the product dataset.
- Verify the Azure SQL product source and the configured ADLS sink.
- This activity copies product master data into the raw data layer.
- After the source activities are configured, the complete ingestion pipeline can be validated and executed.

Step 10: Run and Validate the ADF Pipeline

The screenshot displays the Microsoft Azure Data Factory (ADF) interface. The top navigation bar shows the user is logged in as 'krishnaprasadpm758@gmail.com'. The left sidebar shows the 'Factory Resources' pane with 'Pipelines' and 'Datasets' sections. The 'Pipelines' section shows 'PL_Ingest_Retail_Data' with a status of '1'. The 'Activities' pane shows 'Copy data' and 'Data flow' activities. The main canvas shows a pipeline diagram with four 'Copy data' activities (Copy data1 to Copy data4) connected sequentially. The 'Output' tab is selected, showing the 'Pipeline status' as 'Succeeded'. Below this, a table lists the activities and their execution details.

Activity name	Activity st...	Activit...	Run start	Duration	Integration runtime	User prop...
Copy data4	Succeeded	Copy data	9/25/2026, 11:37:01 AM	23s	AutoResolveIntegrationRuntime (East US)	
Copy data3	Succeeded	Copy data	9/25/2026, 11:36:36 AM	23s	AutoResolveIntegrationRuntime (East US)	
Copy data2	Succeeded	Copy data	9/25/2026, 11:36:07 AM	29s	AutoResolveIntegrationRuntime (East US)	
Copy data1	Succeeded	Copy data	9/25/2026, 11:35:37 AM	29s	AutoResolveIntegrationRuntime (East US)	

Successful pipeline execution

Step-by-step explanation

- Run or debug the completed ingestion pipeline.
- Review the Output section and verify that the Copy Data activities completed successfully.
- A successful run confirms that the source datasets were transferred without an activity failure.
- After the run, verify the landing location in ADLS to confirm that the raw files are available for Databricks processing.

Step 11: Verify the Bronze Data

Microsoft Azure

Search resources, services, and docs (G+/)

Copilot

krishnaprasadpm758@...
DEFAULT DIRECTORY (EVANPUNL...

Home > kpstorage898_1790312486474 | Overview > kpstorage898 | Containers

retail-bronze
Container

Search

◊ <<

+ Add Directory

↑ Upload

↻ Refresh

🗑 Delete

📄 Copy

📄 Paste

🔄 Rename

🔗 Acquire lease

🔗 Break lease

🔗 Edit columns

Overview

Diagnose and solve problems

Access Control (IAM)

Settings

retail-bronze

Authentication method: Access key (Switch to Microsoft Entra user account)

Search blobs by prefix (case-sensitive)

Only show active objects

Showing all 4 items

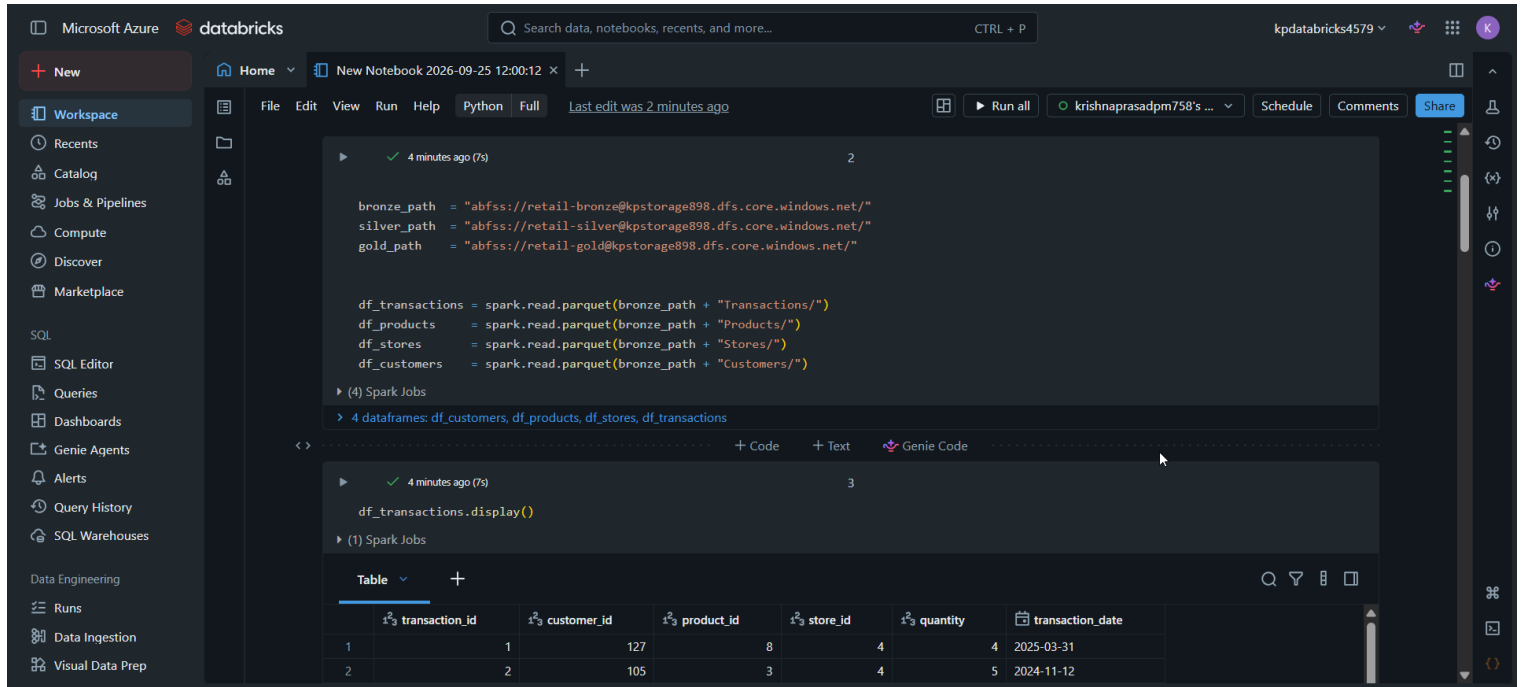
☐	Name	Last modified	Access tier	Blob type	Size	Lease state	
☐	Customers	9/25/2026, 11:37:20 AM	Hot (Inferred)	Block blob	3.17 KiB	Available	...
☐	Products	9/25/2026, 11:36:56 AM	Hot (Inferred)	Block blob	1.21 KiB	Available	...
☐	Stores	9/25/2026, 11:36:32 AM	Hot (Inferred)	Block blob	867 B	Available	...
☐	Transactions	9/25/2026, 11:36:02 AM	Hot (Inferred)	Block blob	1.72 KiB	Available	...

ADLS Bronze/raw layer after ingestion

Step-by-step explanation

- Open the Bronze/raw container in the storage account.
- Verify that the ingested datasets are present in the landing area.
- Check the files and their availability before starting the Databricks transformation stage.
- The Bronze layer acts as the raw-data foundation for the subsequent cleaning and transformation process.

Step 12: Read the Inventory Data in Databricks



```
bronze_path = "abfss://retail-bronze@kpstorage898.dfs.core.windows.net/"
silver_path = "abfss://retail-silver@kpstorage898.dfs.core.windows.net/"
gold_path = "abfss://retail-gold@kpstorage898.dfs.core.windows.net/"

df_transactions = spark.read.parquet(bronze_path + "Transactions/")
df_products = spark.read.parquet(bronze_path + "Products/")
df_stores = spark.read.parquet(bronze_path + "Stores/")
df_customers = spark.read.parquet(bronze_path + "Customers/")

df_transactions.display()
```

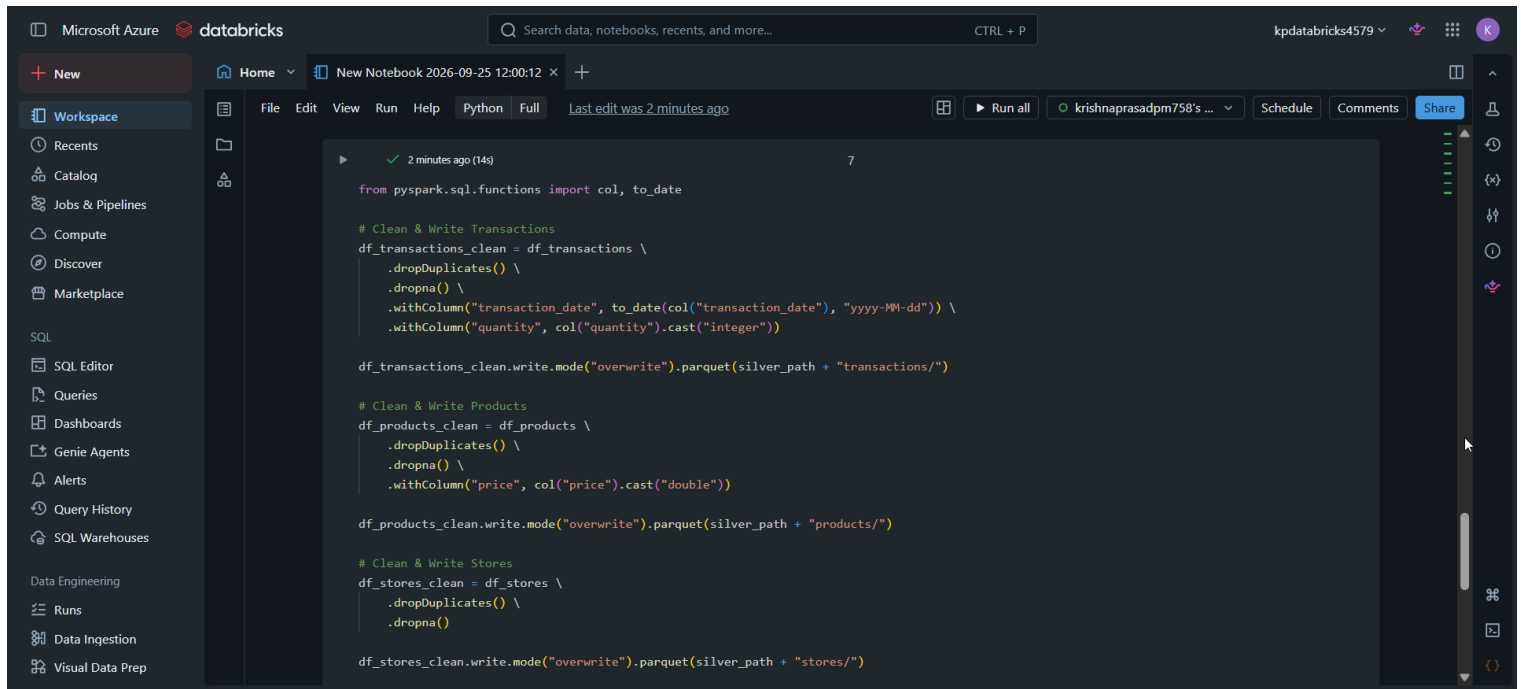
	transaction_id	customer_id	product_id	store_id	quantity	transaction_date
1	1	127	8	4	4	2025-03-31
2	2	105	3	4	5	2024-11-12

Databricks notebook reading the inventory dataset

Step-by-step explanation

- Open the Databricks notebook used for the retail data processing.
- Configure the notebook to access the ADLS data location and read the inventory dataset.
- Display the DataFrame to inspect the available inventory columns and records.
- Use this inspection to understand the raw structure before applying the transformation logic.

Step 13: Transform the Retail Data in Databricks



The screenshot shows a Databricks workspace with a notebook titled "New Notebook 2026-09-25 12:00:12". The notebook is in "Python" mode and shows a code cell that has been executed successfully 2 minutes ago. The code performs the following steps:

- Imports `col` and `to_date` from `pyspark.sql.functions`.
- Creates `df_transactions_clean` by dropping duplicates, dropping nulls, and converting the `transaction_date` column to a date format (`yyyy-MM-dd`).
- Writes `df_transactions_clean` to a Parquet file in the `silver_path` directory, overwriting any existing data.
- Creates `df_products_clean` by dropping duplicates, dropping nulls, and casting the `price` column to a double.
- Writes `df_products_clean` to a Parquet file in the `silver_path` directory, overwriting any existing data.
- Creates `df_stores_clean` by dropping duplicates and dropping nulls.
- Writes `df_stores_clean` to a Parquet file in the `silver_path` directory, overwriting any existing data.

Databricks notebook transformation logic

Step-by-step explanation

- Continue the Databricks processing after the raw datasets have been loaded.
- Apply the notebook transformations required to clean and standardize the retail data.
- Prepare the datasets for the refined Silver layer and subsequent analytical processing.
- The notebook view shows the PySpark transformation code and execution results used in this stage.

Step 14: Verify the Silver Layer

Microsoft Azure

Search resources, services, and docs (G+/)

Copilot

krishnaprasadpm758@...
DEFAULT DIRECTORY (EVANPUNL...

Home > Evan_DE > kpstorage898 | Containers

retail-silver
Container

Search

◊ <<

+ Add Directory

↑ Upload

↻ Refresh

🗑 Delete

📄 Copy

📄 Paste

🔄 Rename

🔗 Acquire lease

🔗 Break lease

🔗 Edit columns

Overview

Diagnose and solve problems

Access Control (IAM)

Settings

retail-silver

Authentication method: Access key (Switch to Microsoft Entra user account)

Search blobs by prefix (case-sensitive)

Only show active objects

Showing all 4 items

☐	Name	Last modified	Access tier	Blob type	Size	Lease state
☐	customers	9/25/2026, 12:16:34 PM				...
☐	products	9/25/2026, 12:16:30 PM				...
☐	stores	9/25/2026, 12:16:32 PM				...
☐	transactions	9/25/2026, 12:16:27 PM				...

Add or remove favorites by pressing Ctrl+L+Shift+F

ADLS Silver/processed layer

Step-by-step explanation

- Open the Silver container after the Databricks processing completes.
- Verify that the processed datasets have been written to the Silver layer.
- Check that the expected retail datasets are present and available for further modelling.
- Silver represents the refined stage of the pipeline after the raw Bronze data has been processed.

Step 15: Build the Curated Sales Dataset

The screenshot shows the Databricks interface with a notebook titled 'Retail data'. The notebook contains a Python cell with the following code:

```
fact_sales = df_transactions \
    .join(df_products, "product_id", "left") \
    .select(
        col("transaction_id"),
        col("customer_id"),
        col("product_id"),
        col("store_id"),
        col("transaction_date"),
        col("quantity"),
        round(col("quantity") * col("price"), 2).alias("revenue")
    )

fact_sales.write.mode("overwrite").parquet(gold_path + "fact_sales/")
```

Below the code cell, the output shows a table view of the data. The table has the following columns: transaction_id, customer_id, product_id, store_id, transaction_date, quantity, and revenue. The revenue column is highlighted with a value of 1.2.

Databricks transformation and sales-data preparation

Step-by-step explanation

- Continue the notebook processing using the refined datasets.
- Apply the required joins, selections, calculations, and transformations to prepare the sales dataset.
- Review the notebook output to confirm that the transformed records are being produced correctly.
- The resulting curated data is prepared for the analytical Gold layer.

Step 16: Validate the Curated Sales Output

Microsoft Azure databricks

Search data, notebooks, recents, and more... CTRL + P

kpdatabricks4579

New

Home

Retail data

Workspace

Recents

Catalog

Jobs & Pipelines

Compute

Discover

Marketplace

SQL

SQL Editor

Queries

Dashboards

Genie Agents

Alerts

Query History

SQL Warehouses

Data Engineering

Runs

Data Ingestion

Visual Data Prep

File Edit View Run Help Python Full Last edit was 2 minutes ago

fact_sales.display()

(2) Spark Jobs

Table

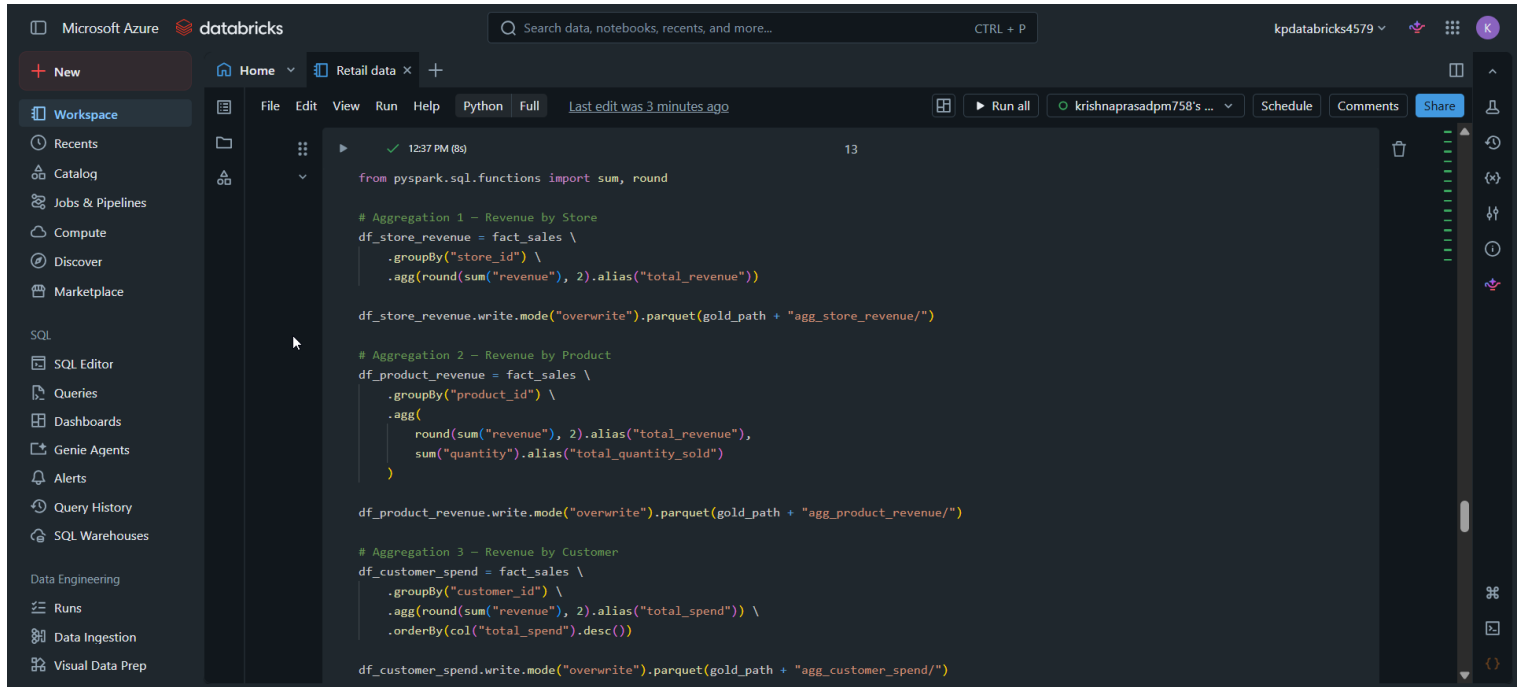
	transaction_id	customer_id	product_id	store_id	transaction_date	quantity	revenue
1	6	110	7	3	2025-01-04	5	24995
2	19	116	8	4	2024-07-14	4	1596
3	16	118	6	2	2024-11-29	4	1196
4	17	109	8	5	2024-07-10	5	1995
5	12	108	5	1	2025-05-26	4	596
6	5	105	5	2	2025-03-17	1	149
7	22	107	9	3	2024-11-16	4	7996
8	24	125	1	5	2024-07-14	1	799.99
9	11	102	1	3	2024-08-11	2	1599.98
10	20	130	5	1	2024-07-30	2	298
11	2	105	3	4	2024-11-12	5	2495
12	26	126	6	2	2024-09-21	2	598
13	27	127	8	1	2024-10-10	1	399
14	15	121	6	5	2025-05-19	5	1495
15	18	103	1	4	2024-09-05	3	2399.97

Databricks table output for the processed sales data

Step-by-step explanation

- Display the transformed sales DataFrame in Databricks.
- Inspect the resulting columns and sample records to confirm that the transformation produced the expected structure.
- Use the output as a validation checkpoint before writing the final analytical datasets.
- This step confirms that the curated sales data is ready for Gold-layer processing.

Step 17: Create Gold-Level Aggregations



The screenshot shows the Databricks interface with a notebook titled 'Retail data'. The notebook contains Python code for three aggregations: Revenue by Store, Revenue by Product, and Revenue by Customer. The code uses PySpark to group data and calculate aggregates, then writes the results to the Gold layer.

```
from pyspark.sql.functions import sum, round

# Aggregation 1 - Revenue by Store
df_store_revenue = fact_sales \
    .groupBy("store_id") \
    .agg(round(sum("revenue"), 2).alias("total_revenue"))

df_store_revenue.write.mode("overwrite").parquet(gold_path + "agg_store_revenue/")

# Aggregation 2 - Revenue by Product
df_product_revenue = fact_sales \
    .groupBy("product_id") \
    .agg(
        round(sum("revenue"), 2).alias("total_revenue"),
        sum("quantity").alias("total_quantity_sold")
    )

df_product_revenue.write.mode("overwrite").parquet(gold_path + "agg_product_revenue/")

# Aggregation 3 - Revenue by Customer
df_customer_spend = fact_sales \
    .groupBy("customer_id") \
    .agg(round(sum("revenue"), 2).alias("total_spend")) \
    .orderBy(col("total_spend").desc())

df_customer_spend.write.mode("overwrite").parquet(gold_path + "agg_customer_spend/")
```

Databricks Gold transformation logic

Step-by-step explanation

- Apply the final aggregation logic in the Databricks notebook.
- Create the business-ready aggregated datasets required for retail analysis.
- Review the generated output and confirm that the aggregation logic has executed successfully.
- Write the resulting curated datasets to the Gold layer.

Retail Data Engineering • Screenshot 17 of 18

Step 18: Verify the Gold Layer

Microsoft Azure

Search resources, services, and docs (G+)

Copilot

krishnaprasadpm758@...
DEFAULT DIRECTORY (EVANPUNL...

Home > kpstorage898 | Containers

retail-gold
Container

Search

Overview

Diagnose and solve problems

Access Control (IAM)

Settings

retail-gold

Authentication method: Access key (Switch to Microsoft Entra user account)

Search blobs by prefix (case-sensitive)

Only show active objects

Showing all 8 items

	Name	Last modified	Access tier	Blob type	Size	Lease state
☐	agg_customer_spend	9/25/2026, 12:38:03 PM				...
☐	agg_date_revenue	9/25/2026, 12:38:04 PM				...
☐	agg_product_revenue	9/25/2026, 12:38:01 PM				...
☐	agg_store_revenue	9/25/2026, 12:38:00 PM				...
☐	dim_customers	9/25/2026, 12:34:06 PM				...
☐	dim_products	9/25/2026, 12:34:04 PM				...
☐	dim_stores	9/25/2026, 12:34:05 PM				...
☐	fact_sales	9/25/2026, 12:35:26 PM				...

Add or remove favorites by pressing Ctrl+L+Shift+F

ADLS Gold/curated layer

Step-by-step explanation

- Open the Gold container after the Databricks processing completes.
- Verify that the final curated datasets have been written successfully.
- Check the available Gold outputs before using the data for downstream analytics.
- This completes the documented ingestion and Medallion-style processing flow from source systems through Bronze, Silver, and Gold.